

QUDA multigrid 算法实现

近零模聚合、Galerkin 粗化与递归 V-cycle 的源码证据链

源码分析汇报

QUDA 1.1.0 参考源码快照

2026 年 8 月 28 日

结论先行: QUDA 的 MG 是递归的自适应预条件器

一条主线

$$B_\ell \longrightarrow V_\ell \longrightarrow (P_\ell, R_\ell)$$

$$D_{\ell+1} = R_\ell D_\ell P_\ell$$

smooth $\longrightarrow$ restrict $\longrightarrow$ coarse solve
 $\longrightarrow$ prolongate $\longrightarrow$ smooth

物理/数值解释

平滑器快速削弱局部高频误差; 由近零模张成的粗空间负责长程、低频误差。每一层重复同一逻辑, 因此最底层可以是粗 Dirac 系统, 而不是特殊的手写矩阵。

源码确证

确证 'MG::operator()' 实现递归 V-cycle;

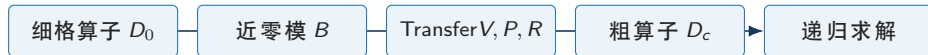
确证 'Transfer' 同时提供 P 与 R ;

确证 UV/VUV 装配粗算子;

确证 'verify()' 比较 RDP 与 native apply;

报告边界

以下结论来自静态源码与测试入口。**未验证** 本次未启动 CUDA/MPI, 因此不宣称具体吞吐、迭代数或硬件收敛曲线。



问题的第一性原理：粗空间要近似难解的低频子空间

误差分解

设误差为 $e = e_{\text{low}} + e_{\text{high}}$ 。局部 smoother 对 e_{high} 有效，但对随尺度缓慢变化的 e_{low} 收敛很慢。若 V 的列覆盖后者，则

$$e_{\text{low}} \simeq P e_c, \quad e_c \simeq (RDP)^{-1} Rr.$$

维数与伴随关系

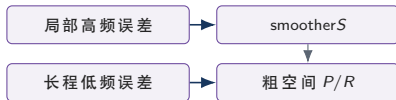
$$P: \mathcal{V}_c \rightarrow \mathcal{V}_f, \quad R: \mathcal{V}_f \rightarrow \mathcal{V}_c,$$

聚合路径中 $R = P^\dagger$ ，因此

$$D_c = RDP, \quad P^\dagger P \simeq I_c.$$

量纲/实现检查

每个细格点必须映射到唯一粗点；每个 aggregate 内的列必须归一；粗算子 apply 必须与显式 $P \rightarrow D \rightarrow R$ 的动作相符。



来源：Galerkin/投影检查：lib/multigrid.cpp:785-928；Transfer：include/transfer.h。

对象与生命周期: API 只负责入口, MG 层级负责状态

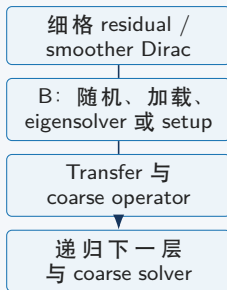
```
loadGaugeQuda( $U$ ,  $C$ )  
mg = newMultigridQuda(mg_param)  
invertQuda( $x$ ,  $b$ ; preconditioner = mg)  
updateMultigridQuda(mg, mg_param)  
dumpMultigridQuda(mg, mg_param)  
destroyMultigridQuda(mg)
```

每层运行期状态

对象	作用
B_ℓ	setup 生成/加载的候选近零模
V_ℓ, P_ℓ, R_ℓ	聚合局部基与双向传输
Y_ℓ, X_ℓ	coarse links 与 local/clover 项
smoother	pre/post 局部误差消除
coarse solver	下一层 MG 或底层 Krylov 包装

来源: API: `include/quda.h:1256-1292`; 构造/更新: `lib/interface_quda.cpp:2853-3053`。

构造顺序



更新的分叉

thin update 复用层级结构、更新引用的 gauge/clover 与质量; full update 重新建细格 Dirac 并递归 `reset(refresh=true)`。

层级 setup 伪代码：先建本层，再递归装配下一层

语言无关等价实现

```
setupLevel( $\ell, D_\ell, B_\ell, \text{cfg}$ )  
  if  $\ell < L - 1$ :  $B_\ell \leftarrow \text{make/load/smooth}(B_\ell)$   
     $T_\ell \leftarrow \text{Transfer}(B_\ell, b_\mu, s_b, N_v)$   
     $(P_\ell, R_\ell) \leftarrow T_\ell$   
     $D_{\ell+1} \leftarrow \text{Galerkin}(D_\ell, T_\ell)$   
     $S_\ell \leftarrow \text{makeSmoother}(D_\ell, \text{cfg}_\ell)$   
     $B_{\ell+1} \leftarrow R_\ell B_\ell$  (或独立生成)  
     $MG_{\ell+1} \leftarrow \text{setupLevel}(\ell + 1,$   
       $D_{\ell+1}, B_{\ell+1})$   
     $C_\ell \leftarrow \text{coarseSolver}(MG_{\ell+1}, S_{\ell+1})$   
    return  $(T_\ell, D_{\ell+1}, S_\ell, C_\ell)$   
  else:  $S_{L-1} \leftarrow \text{bottom solver/smooth}$ 
```

来源：递归装配：lib/multigrid.cpp:72-197；伪代码为控制流等价重写。

generate_all_levels

TRUE: 各层可独立生成 B ;
FALSE: 先做 $B_{\ell+1} = R_\ell B_\ell$,
再重建下一层 Transfer。
这不是内存优化，而是改变
setup 数据流。

顺序约束

createCoarseDirac 发生在
createSmoother 之前；下一
层和 solver 在 coarse operator
建好后才创建。

近零模 setup 伪代码：随机向量只是起点，不是最终粗空间

`'generateNullVectors(B,refresh)'`

$\text{maxiter} \leftarrow \text{refresh} ? \text{setup_maxiter_refresh} : \text{setup_maxiter}$
 $\text{solver} \leftarrow (\text{setup_inv_type}, \text{tol}, \text{precision}, \text{use_init} = \text{YES})$

for $q = 1, \dots, \text{num_setup_iter}$:

 可选：全局 Gram-Schmidt(B)

for each batch $I \subset \{0, \dots, N_v - 1\}$:

if $\text{setup_type} = \text{TEST}$: $(\text{rhs}, x) \leftarrow (B_I, 0)$

else: $(\text{rhs}, x) \leftarrow (0, B_I)$

$(x \leftarrow \text{solve}(D_\ell x = \text{rhs}))$

$B_I \leftarrow x$

 可选：全局 Gram-Schmidt(B)

if solver=MG: reset/rebuild T ，并刷新下一层

来源：setup 参数与 batch: lib/multigrid.cpp:1275-1413; MG 自举: lib/multigrid.cpp:1338-1363; 枚举: include/enum_quda.h:464-482。

关键语义

'TEST' 分支以 B 为右端、零初值；普通 null setup 以 B 为初值、零右端，让迭代器保留慢模而衰减快模。

特殊路径

CG/CA-CG 包装 $D^\dagger D$; MG setup 把自身作为 GCR 的预条件器；quarter halo 在 setup 期间临时提升到 half。

块 Gram–Schmidt: 正交对象是 aggregate-local 的列, 不是全局向量

block orthogonalize 的数学伪代码

for each aggregate a and coarse chirality q :

for $j = 0, \dots, N_v - 1$: $v_j \leftarrow B_j|_{a,q}$

for $i < j$: $\alpha_{ij} = \sum_{x \in a} v_i(x)^\dagger v_j(x)$

$v_j \leftarrow v_j - \alpha_{ij} v_i$

$n_j^2 = \sum_{x \in a} \|v_j(x)\|^2$

$v_j \leftarrow v_j / \sqrt{n_j^2}$

$V_j(a, q) \leftarrow v_j$

repeat n_block_ortho 次; two_pass 用固定点复核/再正交

实现对应

首轮从 B 读入; 后续重复轮次从 V 读入。CUDA kernel 以 coarse aggregate、chirality 和向量 tile 并行化; ‘mVec’ 只是批量化, 不改变 Gram–Schmidt 顺序语义。

必要不变量

对每个 aggregate, $V_a^\dagger V_a \simeq I$ 。
仅做全局正交不能替代这个条件。

来源: kernel 主循环: include/kernels/block_orthogonalize.cuh:122-253; 精度 dispatch: lib/block_orthogonalize.in.cu:42-308; 调用: lib/transfer.cpp:92-112。

Transfer 伪代码：几何 map、spin map 与 P/R 必须共享同一约定

aggregate 路径

$a(x) \leftarrow \text{coarseSite}(x_\mu / b_\mu)$

$s_c \leftarrow \text{spinMap}(s_f, \text{parity})$

P: $\psi_f(x) \leftarrow \sum_{c_c, j} V_j(x) \eta_c(a(x), s_c, c_c, j)$

R: $\eta_c(a) \leftarrow \sum_{x \in a} V(x)^\dagger \psi_f(x)$

$R = P^\dagger$ (aggregate; 同一 V , $a(x)$, spinMap)

索引/边界检查

fine_to_coarse 覆盖每个细点；coarse_to_fine 按 aggregate 排序。block size 要整除局部格点，并受奇偶 coarse 维限制。

变体

aggregate: 真正使用 V ;
coarse-KD: 固定 staggered KD 结构;
optimized-KD: 元数据/等体积映射,
不能直接套用 aggregate 公式。

来源: map/P/R: lib/transfer.cpp:200-328; kernel: include/kernels/prolongator.cuh。

Galerkin 粗算子：快速 kernel 仍必须等价于 RDP

Wilson/Clover 及其变体的装配伪代码

```
exchangeGhost( $V, nFace$ )
     $nFace = 1$  (Wilson/Clover), 3 (ASQTAD)
if preconditioned Clover:  $AV \leftarrow C^{-1}V$ 
for  $\mu = 0, \dots, 3$  and each direction :
     $UV_\mu \leftarrow U_\mu V$  (或  $U_\mu AV$ )
     $Y_\mu += V^\dagger(UV_\mu)$ 
    if long link:  $LV_\mu \leftarrow L_\mu V$ 
     $Y_\mu += V^\dagger(LV_\mu)$ 
if not bidirectional:  $Y_{+\mu} \leftarrow \text{reverse}(Y_{-\mu})$ 
 $X \leftarrow V^\dagger CV + I + \text{twisted-mass/local terms}$ 
return  $D_c = (X \text{ 的 local block}) + (Y \text{ 的 hopping blocks})$ 
```

来源：粗化总流程：lib/coarse_op.cuh:947-1463；UV/VUV kernel：include/kernels/coarse_op_kernel.cuh。

为什么有 UV/VUV

UV 先把 fine link 作用到所有基向量； $V^\dagger(UV)$ 再投影到粗颜色/自旋。这样避免逐列显式构造矩阵，且可在 kernel 中原子累加。

双向链接

普通 Wilson 可单向构造后反推；PC Clover、twisted mass、KD 等路径会强制 bidirectional build。

X、PC 与 coarse-on-coarse: 局部项不能被“只粗化 hopping”替代

local block 的装配

if Clover: $X \leftarrow V^\dagger CV$

else: $X \leftarrow I$

if twisted mass: $X \leftarrow X + \mu$ term

if staggered: $X \leftarrow X +$ staggered mass

if coarse-on-coarse: C 取上一层 coarse clover

预条件 coarse operator

$$D_c = \begin{pmatrix} A_e & D_{eo} \\ D_{oe} & A_o \end{pmatrix}$$

$$\hat{D}_{eo} = A_e^{-1} D_{eo}, \quad \hat{D}_{oe} = A_o^{-1} D_{oe}$$

PC smoother/solver: only one parity enters transfer

full solution: reconstruct the other parity

精度安全

half/fixed-point 路径先求 max 与 scale, 再
atomic accumulate、convert/rescale; scale 是数值协议的一部分, 不是单纯的存储细节。

配置合同

coarse_grid_solution_type=MATPC 要求相应的 preconditioned smoother; full 与 parity 方案不能混用。



递归 V-cycle (1/2): 先明确 solution type 与 residual 来源

进入本层的伪代码

```
outerType  $\leftarrow$  ( $b$  是 full ? MAT : MATPC)
innerType  $\leftarrow$  coarse_grid_solution_type $\ell$ 
uniform  $\leftarrow$  (smooth solve type = innerType)
prepare( $out, in, x, b, outerType$ )
if presmoothing: ( $out, in$ )  $\leftarrow S_{pre}(out, in)$ 
if not uniform:  $x \leftarrow$  reconstruct( $x, b, innerType$ )
if need explicit residual:  $r \leftarrow b - Dx$ 
residual  $\leftarrow$  solver residual /  $in/b/r$  按分支选择
```

为什么不能只写

$$r = b - Dx$$

预条件 smoother 可能只在单奇偶子格工作。若 solver 与 coarse solution type 不一致, 必须先 reconstruct, 再显式用 residual Dirac 重算 r 。

非法组合

MATPC coarse solution 要求 preconditioned smoother; 源码在运行前直接报错。

来源: 运行期入口与 uniform/residual 分支: lib/multigrid.cpp:1131-1183; solution type 设定: tests/utils/set_params.cpp:555-640。

递归 V-cycle (2/2): 限制、粗解、校正、后平滑

按源码控制流展开

```
 $r_c \leftarrow R_\ell$  residual  
 $e_c \leftarrow C_\ell(r_c)$  (下一层 MG 或底层 Krylov)  
if no presmoothener:  $x \leftarrow x + P_\ell e_c$   
else:  $\delta \leftarrow P_\ell e_c$ ;  $x \leftarrow x + \delta$   
if not uniform: prepare(out, in, x, b, innerType)  
if postsmoothener: (out, in)  $\leftarrow S_{post}(out, in)$   
 $x \leftarrow \text{reconstruct}(x, b, \text{outerType})$   
return x  
bottom level: prepare  $\rightarrow$  pre-smooth/solve  $\rightarrow$  reconstruct
```

残差与校正的关键

有 pre-smoothen 时, $P e_c$ 要加到已经平滑的 x ; 源码把校正写入临时 residual, 再 'xpy' 累加, 避免覆盖平滑解。

来源: V-cycle: lib/multigrid.cpp:1131-1224; coarse solver: lib/multigrid.cpp:535-702。

数学闭环

$$x \xrightarrow{S_{pre}} x_s \xrightarrow{R(b - D x_s)} e_c \xrightarrow{P} x_s + P e_c \xrightarrow{S_{post}} x^+.$$

coarse solver 与 cycle: V-cycle 不是唯一的“求解器对象”

wrapper 语义

coarse solver = PreconditionedSolver(solver,
MG/smoother)
solver $\in \{\text{GCR, CGNR/CGNE, BiCGStabL, CA variants, ...}\}$
next level MG $\Rightarrow$ one recursive
preconditioner application
bottom level $\Rightarrow$ Krylov solve/deflation as configured

当前快照实际支持的分支

VCYCLE: 中间层可直接复用 coarse MG;
RECURSIVE: 创建 solver wrapper;
FCYCLE/WCYCLE 枚举存在, 但本实现
工厂未走通用分支。
不能只凭枚举名声称已实现完整 F/W cycle。

外层关系

MG 作为外层 Krylov 的 preconditioner 时, 一次 MG 调用是 $K(r)$; 粗层精度、Schwarz 和 setup 状态使其表现为可变预条件器。

来源: cycle/wrapper: lib/multigrid.cpp:535-702; 枚举: include/enum_quda.h:178-184。

实现层：halo、checkerboard、低精度与 MMA 只改变 “如何算”

粗算子/粗 dslash 的后端伪代码

```
exchangeGhost(V, depth = nFace)
for direction  $\mu$ :  load local/ghost coarse spinor
     $y \leftarrow Y_\mu \text{spinor}_{x+\hat{\mu}} + Y_\mu^\dagger \text{spinor}_{x-\hat{\mu}}$ 
     $y \leftarrow y + X \text{spinor}_x$ 
bulk kernel || halo kernel;  parity/checkerboard separate
```

优化不变量

global/shared atomic、parity flip、coarse-color wave、autotuning、MMA/tensor-core 只替换 tile 和调度； $V^\dagger$ 、方向共轭转置和 $R = P^\dagger$ 不能变。

来源：粗算子 kernel: lib/coarse_op.cuh:221-915; coarse Dirac: lib/dirac_coarse.cpp:121-479。

开关/字段	影响
'location'	每层 CPU/GPU 放置
setup_location	null/setup 与粗算子构造位置
precision_null	B/V 存储精度
smoother_halo_precision	smoother halo 精度
setup/dslash/transfer_use_mma	三类 kernel 的 MMA 选择
fixed-point	max/scale/convert 协议

参数合同：最少要同步几何、零空间、solver 与 solution type

参数组	代表字段	实现含义
层级/几何	n_level、geo_block_size、spin_block_size	粗格体积、aggregate 容量、spin/chirality map；block 需满足整除和 parity 约束。
零空间	n_vec、n_vec_batch、setup_type	列数、批处理和 TEST/null setup 分支；batch 不整除直接失败。
setup	setup_inv_type、num_setup_iter、setup_tol	生成/刷新 B 的迭代器、次数与停机目标。
Transfer	transfer_type、precision_null	aggregate、KD 或 optimized-KD；决定是否有实际 V。
smooth	smoother、nu_pre/post、omega、Schwarz	pre/post 应用次数、松弛与局部预条件。
coarse	coarse_solver、coarse_grid_solution_type	下一层 wrapper、底层 Krylov，以及 full/PC residual 语义。

一个容易漏掉的约束

generate_all_levels=false 且计算 null vectors 时，各层 n_vec 必须一致；否则参数检查拒绝配置。 MATPC solution 又要求 preconditioned smoother。

来源：参数结构/检查：include/quda.h:638-836；lib/check_params.h:930-1075。

验证矩阵：把“粗化正确”拆成可定位的不变量

验证伪代码

```
for null vector  $B_i$ :  $e_i \leftarrow B_i - P(RB_i)$ 
for coarse random  $\eta$ :  $z \leftarrow \eta - R(P\eta)$ 
for coarse random  $\eta$ :  $z \leftarrow D_c\eta - R(D(P\eta))$ 
if PC:  $z \leftarrow \hat{D}_c\eta - A_c^{-1}D_c\eta$ 
check normality:  $\text{Im}\langle\eta, M^\dagger M\eta\rangle \simeq 0$ 
optional: low-mode overlap / oblique projection
fail if relative L2 or max deviation exceeds tolerance
```

失败项	优先回查
$PRB \neq B$	block ortho、map、 V 精度
$RP\eta \neq \eta$	R/P spin/parity 与归一化
$RDP \neq D_c$	UV/VUV、方向、local X
PC 误差大	$C^{-1}V$ 、Schur parity、重构
normality 失败	dagger/halo/粗 dslash

源码中的诚实边界

ASQTAD 的三跳 long link 在 KD 或 aggregate 太小时会主动跳过部分 verify；这应记录为“验证条件不满足”，不能记成通过。

来源：verify: lib/multigrid.cpp:785-1129；ASQTAD 条件：lib/multigrid.cpp:883-928。

测试与动态演化：源码提供回归入口，但不替代本次运行证据

multigrid_evolve_test

```
set defaults → parse MG options → load  $U, C$   
mg ← newMultigridQuda(mg_param)  
invert( $x, b$ )  
for gauge step:  $U \leftarrow$  Monte/reunitarize  
    load  $U, C$ ; updateMultigridQuda(mg)  
    invert( $x, b$ )  
for mass step:  $(m, \kappa, \mu) \leftarrow$  new values  
    update MG → invert
```

来源：演化测试：tests/multigrid_evolve_test.cpp:308-452；benchmark：tests/multigrid_benchmark_test.cpp。

multigrid_benchmark_test

初始化 'DiracCoarse' 与 'DiracCoarsePC'，可对 Dslash、Mat、Clover、 $M^\dagger$ 、 $M^\dagger M$ 及 PC 变体做 gflop/s 测量；还可由 gtest 检查动作。

本报告的验证状态

确证 已核对测试控制流与参数默认值。

未验证 未执行编译后的 CUDA/MPI 测试，故没有把源码中的测试能力写成实测结果。

从零复现的最短路径：先做 golden reference，再开优化

推荐的逐级实现顺序

0. 约定：固定 lattice ordering、parity、spin/color、边界条件和质量归一化
1. 单 rank：随机 $B \rightarrow \text{aggregate-local GS} \rightarrow V \rightarrow (P, R)$
2. Golden coarse： $D_c \eta \equiv R[D(P\eta)]$ 逐粗基/逐动作实现
3. 快速 coarse：替换为 UV/VUV、local X，逐动作比较 golden
4. 递归 solve：加入 pre/post smoother、residual 分支与 bottom solver
5. PC/MPI：加入 Schur prepare/reconstruct、ghost、双向 link 和 halo 精度
6. 性能：half/fixed-point、atomic、MMA、autotune；每步保留 verify

每次实验记录

lattice/global partition、n_vec、block size、
setup solver、smoother、cycle、solution type、
各精度、verify deviation、迭代数与时间。

来源：实现依据：lib/transfer.cpp:200-328、lib/coarse_op.cuh:1043-1463、lib/multigrid.cpp:1131-1224。该页是面向复现的工程排序。

风险排序

先排除数学不一致 ($P/R/D_c$)，再排除 checkerboard/halo，最后才讨论低精度和 kernel 性能；否则性能优化会掩盖粗化错误。

最终结论与源码索引

一句话结论

QUDA 的 multigrid 不是一个固定的“粗网格矩阵”：它是由 setup 产生的局部正交近零模、Galerkin coarse operator、PC/Schur 约定、递归 solver wrapper 和 GPU/MPI 数据路径共同构成的状态机。

$B \rightarrow V \rightarrow RDP \rightarrow$ recursive V-cycle
verify ($PRB, RP_\eta, RDP_\eta, PC, normality$)
only then enable half/fixed-point/MMA/halo
optimizations

主题	主要源码
层级/setup	lib/multigrid.cpp
P/R 与 map	lib/transfer.cpp, include/transfer.h
block GS	include/kernels/ block_orthogonalize.cuh
coarse operator	lib/coarse_op.cuh
coarse Dirac/PC	lib/dirac_coarse.cpp
API/参数/测试	lib/interface_quda.cpp, include/quda.h, tests/

报告完成标准：源码控制流可追踪，伪代码可迁移，验证边界已显式标注。

来源：综合依据：lib/multigrid.cpp; lib/transfer.cpp; lib/coarse_op.cuh。